

BHEESHMA L
192511332

SIMATS ENGINEERING

ASSESSMENT TOOL 3 - LOW (ASSIGNMENT - 2)

COURSE CODE: CSA0921

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO3: *Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 1

Total Marks: 50

ANNEXURE A

ASSIGNMENT - 2

Code of Conduct:

*I Bheeshma L / 192511332 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

BHEESHMA L

ANNEXURE

ASSIGNMENT – 2

1: Online Examination Processing System

A university conducts online examinations in which thousands of students submit answers simultaneously. The examination server must process student submissions while handling invalid student IDs, missing answers, invalid question numbers, duplicate submissions, and unexpected runtime errors. Since submissions arrive concurrently, the system should use multiple threads. The examination system must also ensure that marks are updated correctly when several threads process student records at the same time.

Develop a Java application to:

- a) Create classes for Student, Question, Answer, and Examination.
- b) Use String and numeric validation to process student responses.
- c) Handle exceptions such as `NumberFormatException`, `ArrayIndexOutOfBoundsException`, `NullPointerException`, and `IllegalArgumentException`.
- d) Create a user-defined exception named `InvalidSubmissionException`.
- e) Use `try-catch-finally` to safely process each student's submission.
- f) Create multiple threads to process submissions from different students concurrently.
- g) Use **synchronization** when updating the common marks database.
- h) Demonstrate **thread priorities** by assigning higher priority to examination submission threads nearing the deadline.
- i) Implement suitable handling for uncaught exceptions and generate a final result report after all threads complete.

BHEESHMA L
192511332

```
import java.util.*;
```

```
import java.util.concurrent.ConcurrentHashMap;
```

```
public class OnlineExamSystem {
```

```
    public static void main(String[] args) {
```

```
        Thread.setDefaultUncaughtExceptionHandler((thread, exception) -> {
```

```
            System.err.println("UNCAUGHT EXCEPTION in thread " + thread.getName() + ": " +  
+ exception.getMessage());
```

```
        });
```

```
        Examination exam = new Examination();
```

```
        // Student 1: Perfect submission (Standard priority)
```

```
        List<Answer> answers1 = Arrays.asList(  
            new Answer("1", "A"), new Answer("2", "C"), new Answer("3", "B")
```

```
        );
```

```
        SubmissionProcessor t1 = new SubmissionProcessor("STU-001", answers1, exam,  
false);
```

```
        // Student 2: Near Deadline, valid answers (High priority)
```

```
        List<Answer> answers2 = Arrays.asList(  
            new Answer("1", "A"), new Answer("2", "A") // Q2 is wrong, should get 10 marks
```

```
only
```

```
        );
```

```
        SubmissionProcessor t2 = new SubmissionProcessor("STU-002", answers2, exam, true);
```

BHEESHMA L
192511332

// Student 3: Triggers NumberFormatException

List<Answer> answers3 = Arrays.asList(

new Answer("One", "A")

);

SubmissionProcessor t3 = new SubmissionProcessor("STU-003", answers3, exam,
false);

// Student 4: Triggers InvalidSubmissionException (Empty answer)

List<Answer> answers4 = Arrays.asList(

new Answer("1", " ")

);

SubmissionProcessor t4 = new SubmissionProcessor("STU-004", answers4, exam,
false);

// Student 5: Triggers ArrayIndexOutOfBoundsException

List<Answer> answers5 = Arrays.asList(

new Answer("99", "C")

);

SubmissionProcessor t5 = new SubmissionProcessor("STU-005", answers5, exam,
false);

// Start all threads (f)

t1.start();

t2.start();

t3.start();

t4.start();

t5.start();

```
// Wait for all threads to finish before generating the report

try {

    t1.join();

    t2.join();

    t3.join();

    t4.join();

    t5.join();

} catch (InterruptedException e) {

    System.err.println("Main thread interrupted.");

}


// i) Generate a final result report after all threads complete.

exam.printFinalReport();

}

}


// d) Create a user-defined exception named InvalidSubmissionException.

class InvalidSubmissionException extends Exception {

    public InvalidSubmissionException(String message) {

        super(message);

    }

}


// a) Create classes for Student, Question, Answer, and Examination.
```

BHEESHMA L
192511332

```
class Student {  
  
    String id;  
  
    int marks;  
  
  
    public Student(String id) {  
  
        this.id = id;  
  
        this.marks = 0;  
  
    }  
  
  
    @Override  
    public String toString() {  
  
        return "Student ID: " + id + " | Total Marks: " + marks;  
  
    }  
}  
  
  
class Question {  
  
    int questionNumber;  
  
    String expectedAnswer;  
  
  
    public Question(int questionNumber, String expectedAnswer) {  
  
        this.questionNumber = questionNumber;  
  
        this.expectedAnswer = expectedAnswer;  
  
    }  
}
```

BHEESHMA L
192511332

```
class Answer {  
  
    String questionNumberStr;  
  
    String studentAnswer;  
  
    public Answer(String questionNumberStr, String studentAnswer) {  
  
        this.questionNumberStr = questionNumberStr;  
  
        this.studentAnswer = studentAnswer;  
  
    }  
}
```

```
class Examination {  
  
    // Shared database for marks  
  
    private final Map<String, Student> studentDatabase = new ConcurrentHashMap<>();  
  
    private final Question[] questionBank;  
  
    public Examination() {  
  
        // Initializing a mock question bank with 3 questions  
  
        questionBank = new Question[]{  
  
            new Question(1, "A"),  
  
            new Question(2, "C"),  
  
            new Question(3, "B")  
  
        };  
    }  
  
    // g) Use synchronization when updating the common marks database.
```

BHEESHMA L
192511332

```
public synchronized void updateMarks(String studentId, int marksToAdd) {

    studentDatabase.putIfAbsent(studentId, new Student(studentId));

    Student student = studentDatabase.get(studentId);

    student.marks += marksToAdd;

    System.out.println("[DB Sync] Updated marks for " + studentId + ". Current Total: " +
student.marks);

}

public void printFinalReport() {

    System.out.println("\n--- Final Examination Result Report ---");

    for (Student s : studentDatabase.values()) {

        System.out.println(s);

    }

}

public Question[] getQuestionBank() {

    return questionBank;

}

}

// f) Create multiple threads to process submissions from different students concurrently.

class SubmissionProcessor extends Thread {

    private String studentId;

    private List<Answer> answers;

    private Examination exam;

    private boolean isNearDeadline;
```


BHEESHMA L
192511332

```
public SubmissionProcessor(String studentId, List<Answer> answers, Examination exam,  
boolean isNearDeadline) {
```

```
    this.studentId = studentId;
```

```
    this.answers = answers;
```

```
    this.exam = exam;
```

```
    this.isNearDeadline = isNearDeadline;
```

```
    // h) Demonstrate thread priorities by assigning higher priority to threads nearing the  
    deadline.
```

```
    if (isNearDeadline) {
```

```
        this.setPriority(Thread.MAX_PRIORITY);
```

```
    } else {
```

```
        this.setPriority(Thread.NORM_PRIORITY);
```

```
    }
```

```
}
```

```
@Override
```

```
public void run() {
```

```
    System.out.println("Processing started for Student: " + studentId + " (Priority: " +  
this.getPriority() + ")");
```

```
    // e) Use try-catch-finally to safely process each student's submission.
```

```
    try {
```

```
        // b) Use String and numeric validation to process student responses.
```

```
        if (studentId == null || studentId.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException("Student ID cannot be null or empty.");
```

```
        }
```

```
int score = 0;

for (Answer ans : answers) {

    // b) String validation

    if (ans.studentAnswer == null) {

        throw new NullPointerException("Answer string is null for Student " +
studentId);

    }

    if (ans.studentAnswer.trim().isEmpty()) {

        // d) Using the user-defined exception for missing answers

        throw new InvalidSubmissionException("Empty answer submitted by Student "
+ studentId);

    }


    // c) Handle NumberFormatException (Parsing String to int)

    int qNo = Integer.parseInt(ans.questionNumberStr);


    // c) Handle ArrayIndexOutOfBoundsException

    if (qNo < 1 || qNo > exam.getQuestionBank().length) {

        throw new ArrayIndexOutOfBoundsException("Invalid question number: " +
qNo);

    }


    // Evaluate answer (0-indexed array vs 1-indexed question numbers)

    Question q = exam.getQuestionBank()[qNo - 1];

    if (q.expectedAnswer.equalsIgnoreCase(ans.studentAnswer.trim())) {

        score += 10; // Award 10 marks per correct answer
```

BHEESHMA L
192511332

```
    }  
}  
  
    // g) Synchronized update  
    exam.updateMarks(studentId, score);  
  
    // c) Catching the specific exceptions requested  
    } catch (NumberFormatException ex) {  
        System.err.println("Error for " + studentId + ": Invalid numeric format in question  
number - " + ex.getMessage());  
    } catch (ArrayIndexOutOfBoundsException ex) {  
        System.err.println("Error for " + studentId + ": Question out of bounds - " +  
ex.getMessage());  
    } catch (NullPointerException ex) {  
        System.err.println("Error for " + studentId + ": Missing data - " + ex.getMessage());  
    } catch (IllegalArgumentException ex) {  
        System.err.println("Error for " + studentId + ": Invalid Argument - " +  
ex.getMessage());  
    } catch (InvalidSubmissionException ex) {  
        System.err.println("Error for " + studentId + ": Submission Invalid - " +  
ex.getMessage());  
    } finally {  
        // e) Finally block execution  
        System.out.println("Processing completed/terminated for Student: " + studentId);  
    }  
}  
}
```

BHEESHMA L
192511332

Output:

```
Processing started for Student: STU-005 (Priority: 5)
Processing started for Student: STU-001 (Priority: 5)
Processing started for Student: STU-003 (Priority: 5)
Processing started for Student: STU-004 (Priority: 5)
Processing started for Student: STU-002 (Priority: 10)
Error for STU-003: Invalid numeric format in question number - For input string: "One"
Processing completed/terminated for Student: STU-003
Error for STU-004: Submission Invalid - Empty answer submitted by Student STU-004
[DB Sync] Updated marks for STU-001. Current Total: 30
Processing completed/terminated for Student: STU-001
Error for STU-005: Question out of bounds - Invalid question number: 99
[DB Sync] Updated marks for STU-002. Current Total: 10
Processing completed/terminated for Student: STU-002
Processing completed/terminated for Student: STU-004
Processing completed/terminated for Student: STU-005

--- Final Examination Result Report ---
Student ID: STU-002 | Total Marks: 10
Student ID: STU-001 | Total Marks: 30

...Program finished with exit code 0
Press ENTER to exit console.
```